

6

Mining Data Streams

LEARNING OBJECTIVES

After reading this chapter, you will be able to:

- Learn and understand the concept of a data stream.
- Become aware of several real-life data stream applications.
- Understand the standard model for a stream-based management system and be able to identify the issues and challenges a stream model presents.
- Understand the importance of sampling and get introduced to several stream sampling techniques.
- Get introduced to the Bloom Filter and its potential applications.
- Learn about several popular stream-based algorithms like Counting Distinct Elements in a Stream, Counting Ones in a Window.
- Learn the Datar–Gionis–Indyk–Motwani algorithm and how it can be used for query processing in a stream.

6.1 Introduction

In the recent years, a new class of data-intensive applications has become widely recognized, that is, applications in which the data is modeled best not as persistent relational databases but rather as transient *data streams*. Data stream real-time analytics are needed to manage the data currently generated, at an ever increasing rate from these applications. Examples of such applications include financial applications, network monitoring, security, telecommunications data management, web applications, manufacturing, sensor networks, call detail records, email, blogging, twitter posts and others.

In the data stream model, individual data items may be relational tuples, for example, network measurements, call records, web page visits, sensor readings and so on. However, their continuous arrival in multiple, rapid, time-varying, possibly unpredictable and unbounded streams appear to yield some fundamentally new research problems.

In all of the applications cited above, it is not feasible to simply load the arriving data into a traditional database management system (DBMS) and operate on it there. Traditional DBMSs are not designed for rapid and continuous loading of individual data items, and they do not directly support

the *continuous queries* that are typical of data stream applications. Further, the data in a data stream is lost forever if it is not processed immediately or stored. Moreover, the data in most data streams arrive so rapidly that it is not feasible to store it all in active storage (i.e., in a conventional database), and then interact with it at the time of our choice.

Thus, algorithms that process them must do so under very strict constraints of space and time. Consequently, data streams pose several challenges for data mining algorithm design. First, algorithms must make use of limited resources (time and memory). Second, they must deal with data whose nature or distribution changes over time.

6.2 Data Stream Management Systems

Traditional relational databases store and retrieve records of data that are static in nature. Further these databases do not perceive a notion of time unless time is added as an attribute to the database during designing the schema itself. While this model was adequate for most of the legacy applications and older repositories of information, many current and emerging applications require support for online analysis of rapidly arriving and changing data streams. This has prompted a deluge of research activity which attempts to build new models to manage streaming data. This has resulted in data stream management systems (DSMS), with an emphasis on continuous query languages and query evaluation.

We first present a generic model for such a DSMS. We then discuss a few typical and current applications of the data stream model.

6.2.1 Data Stream Model

A data stream is a real-time, continuous and ordered (implicitly by arrival time or explicitly by time-stamp) sequence of items. It is not possible to control the order in which the items arrive, nor it is feasible to locally store a stream in its entirety in any memory device. Further, a query over streams will actually run continuously over a period of time and incrementally return new results as new data arrives. Therefore, these are known as long-running, continuous, standing and persistent queries.

As a result of the above definition, we have the following characteristics that must be exhibited by any generic model that attempts to store and retrieve data streams.

1. The data model and query processor must allow both order-based and time-based operations (e.g., queries over a 10 min moving window or queries of the form which are the most frequently occurring data before a particular event and so on).
2. The inability to store a complete stream indicates that some approximate summary structures must be used. As a result, queries over the summaries may not return exact answers.
3. Streaming query plans must not use any operators that require the entire input before any results are produced. Such operators will block the query processor indefinitely.

4. Any query that requires backtracking over a data stream is infeasible. This is due to the storage and performance constraints imposed by a data stream. Thus any online stream algorithm is restricted to make only one pass over the data.
5. Applications that monitor streams in real-time must react quickly to unusual data values. Thus, long-running queries must be prepared for changes in system conditions any time during their execution lifetime (e.g., they may encounter variable stream rates).
6. Scalability requirements dictate that parallel and shared execution of many continuous queries must be possible.

An abstract architecture for a typical DSMS is depicted in Fig. 6.1. An input monitor may regulate the input rates, perhaps by dropping packets. Data are typically stored in three partitions:

1. Temporary working storage (e.g., for window queries).
2. Summary storage.
3. Static storage for meta-data (e.g., physical location of each source).

Long-running queries are registered in the query repository and placed into groups for shared processing. It is also possible to pose one-time queries over the current state of the stream. The query processor communicates with the input monitor and may re-optimize the query plans in response to changing input rates. Results are streamed to the users or temporarily buffered.

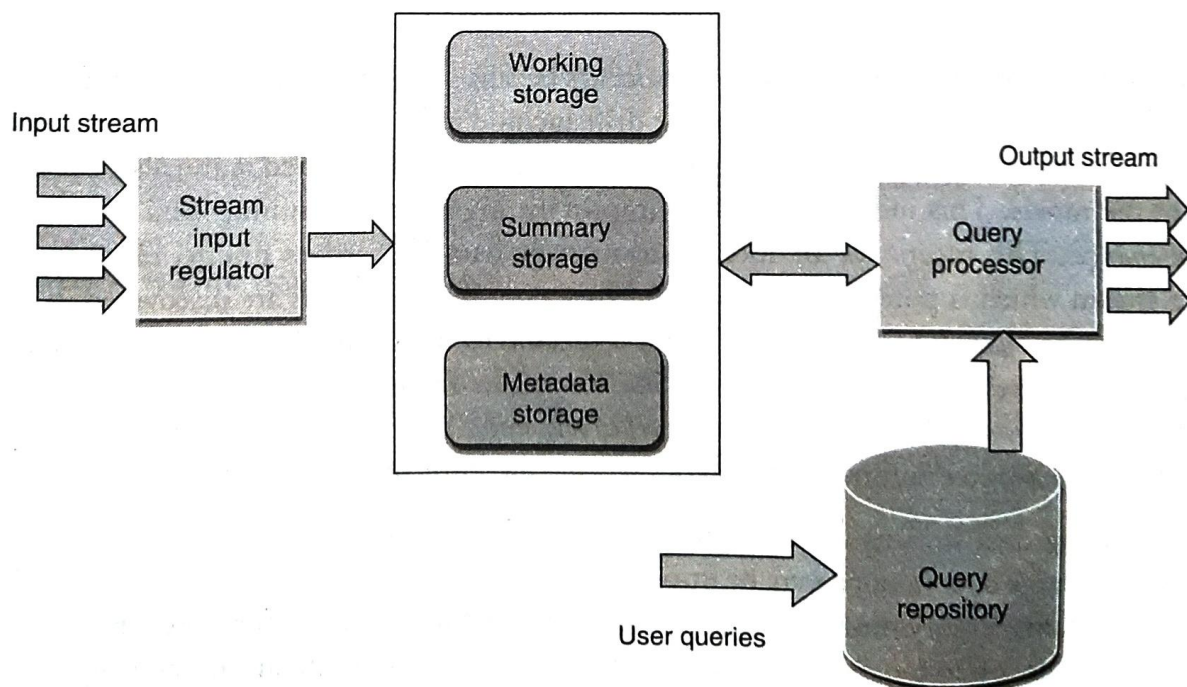


Figure 6.1 Abstract architecture for a typical DSMS.

6.3 Data Stream Mining

Data Stream Mining is the process of extracting useful knowledge from continuous, rapid data streams. Many traditional data mining algorithms can be recast to work with larger datasets, but they cannot address the problem of a continuous supply of data. For example, if a traditional algorithm has learnt and induced a model of the data seen until now, it cannot immediately update the model when new information keeps arriving at continuous intervals. Instead, the entire training process must be repeated with the new examples included.

With big data, this limitation is both undesirable and highly inefficient. Mining big data streams faces three principal challenges: *volume*, *velocity* and *volatility*. Volume and velocity require a high volume of data to be processed in limited time. Starting from the first arriving instance, the amount of available data constantly increases from zero to potentially infinity. This requires incremental approaches that incorporate information as it becomes available, and online processing if not all data can be kept. Volatility, on the other hand, indicates that environment is never stable and has constantly changing patterns. In this scenario, old data is of limited use, even if it could be saved and processed again later. The constantly changing patterns can affect the induced data mining models in multiple ways: *changes in the class label of an existing data variable*, *change in the available feature information*. Both these lead to a phenomenon called *Concept drift*. For example, in a stock market application which labels a particular stock as “hold” or “sell” can change the labels rapidly based on a current stream of input information. Changes in the available feature information can arise when new features become available; for example, a continuous weather forecasting application may now need to consider more attributes because of adding new sensors continuously. Similarly, existing features might need to be excluded due to regulatory requirements, or a feature might change in its scale, if data from a more precise instrument becomes available.

Thus, **Concept drift** is a phenomenon that occurs because of feature changes or changes in behavior of the data itself. Good examples of concept drift include sudden changes in popularity of a movie several days after its release due to good reviews from those who have watched it and also due to changes in price of the movie. This indicates that one important ingredient to mining data streams is *online mining of changes*. This means we are looking to manage data that arrives online, often in real-time, forming a stream which is potentially infinite. Further even if the patterns are discovered in snapshots of data streams the *changes to the patterns* may be more critical and informative. With data streams, people are often interested in mining queries like “*compared to the past few days, what are the distinct features of the current status?*” and “*what are the relatively stable factors over time?*” Clearly, to answer the above queries, we have to examine the changes.

Further mining data streams is challenging in the following two respects. On one hand, random access to fast and large data streams may be impossible. Thus, multi-pass algorithms (i.e., ones that load data items into main memory multiple times) are often infeasible. On the other hand, the exact answers from data streams are often too expensive. The core assumption of data stream processing is that training examples can be briefly inspected a single time only, that is, they arrive in a high speed stream, and then must be discarded to make room for subsequent data. The algorithm processing the stream has no

control over the order of the data seen, and must update its model incrementally as each data element is inspected. Another desirable property is that we must be able to apply the algorithm at any point of time even in between successive arrivals of data elements in the stream.

All these challenges have motivated a whole branch of research, resulting in a new set of algorithms written exclusively for data streams. These algorithms can naturally cope with very large data sizes and can tackle challenging real-time applications not previously tackled by traditional data mining. The most common data stream mining tasks are clustering, classification and frequent pattern mining.

6.4 Examples of Data Stream Applications

We begin by presenting a set of representative data stream applications. This helps us to identify and define a set of query types that a DSMS should support. The reader is referred to the references for many such examples.

6.4.1 Sensor Networks

Sensor networks are a huge source of data occurring in streams. They are used in numerous situations that require constant monitoring of several variables, based on which important decisions are made. In many cases, alerts and alarms may be generated as a response to the information received from a series of sensors. To perform such analysis, aggregation and joins over multiple streams corresponding to the various sensors are required. Some representative queries include the following:

1. Perform a join of several data streams like temperature streams, ocean current streams, etc. at weather stations to give alerts or warnings of disasters like cyclones and tsunamis. It can be noted here that such information can change very rapidly based on the vagaries of nature.
2. Constantly monitor a stream of recent power usage statistics reported to a power station, group them by location, user type, etc. to manage power distribution efficiently.

6.4.2 Network Traffic Analysis

Network service providers can constantly get information about Internet traffic, heavily used routes, etc. to identify and predict potential congestions. Streams of network traffic can also be analyzed to identify potential fraudulent activities. For example, consider the case of an intrusion detection system. Here also it may be the case that a situation may drastically change in a limited time. For example, if a particular server on the network becomes a victim of a denial of service attack, that route can become heavily congested within a short period of time.

Example queries may include the following:

1. Check whether a current stream of actions over a time window are similar to a previous identified intrusion on the network.

2. Similarly check if several routes over which traffic is moving has several common intermediate nodes which may potential indicate a congestion on that route.

6.4.3 Financial Applications

Online analysis of stock prices and making hold or sell decisions requires quickly identifying correlations and fast changing trends and to an extent forecasting future valuations as data is constantly arriving from several sources like news, current stock movements, etc. The following are the typical queries:

1. Find all stocks priced between \$50 and \$200, which is showing very large buying in the last one hour based on some federal bank news about tax cuts for a particular industry.
2. Find all stocks trading above their 100 day moving average by more than 10% and also with volume exceeding a million shares.

6.4.4 Transaction Log Analysis

Online mining of web usage logs, telephone call records and automated bank machine transactions are all examples of data streams since they continuously output data and are potentially infinite. The goal is to find interesting customer behavior patterns, identify suspicious spending behavior that could indicate fraud, etc. The following are some examples:

1. Examine current buying pattern of users at a website and potentially plan advertising campaigns and recommendations.
2. Continuously monitor location, average spends etc of credit card customers and identify potential frauds.

6.5 Stream Queries

Queries over continuous data streams have much in common with queries in a traditional DBMS. Two types of queries can be identified as typical over data streams:

1. **One-time queries:** One-time queries are queries that are evaluated once over a point-in-time snapshot of the data set, with the answer returned to the user. For example, a stock price checker may alert the user when a stock price crosses a particular price point.
2. **Continuous queries:** Continuous queries, on the other hand, are evaluated continuously as data streams continue to arrive. The answer to a continuous query is produced over time, always reflecting the stream data seen so far. Continuous query answers may be stored and updated as new data arrives, or they may be produced as data streams themselves.

Typically aggregation queries, such as finding maximum, average, count, etc., are continuous queries where values are stored. For example, the maximum price of a particular stock every hour. We can answer this query by retaining a simple summary: the maximum of all stream elements ever seen up to now. It is not necessary to record the entire stream. When a new stream element arrives, we compare it with the stored maximum, and set the maximum to whichever is larger. We can then answer the query by producing the current value of the maximum. Similarly, if we want the average price of a stock over all time, we have only to record two values: the number of readings ever sent in the stream and the sum of those readings. We can adjust these values easily each time a new reading arrives, and we can produce their quotient as the answer to the query.

Another class of queries may be join queries that are monotonic and may produce rapid, unbounded answers, dictating the stream approach.

We can also make another distinction in stream queries, that between *pre-defined queries* and *ad-hoc queries*. A pre-defined query is one that is supplied to the DSMS before any relevant data has arrived. Pre-defined queries are most commonly continuous queries.

On the other hand, ad-hoc is issued online after the data streams have already begun. Ad-hoc queries can be either one-time queries or continuous queries. Ad-hoc queries are basically questions asked once about the current state of a stream or streams. If we do not store all streams in their entirety, it is difficult to answer arbitrary queries about streams. If we have some idea what kind of queries will be asked, then we can prepare for them by storing appropriate parts or summaries of streams. Ad-hoc queries complicate the design of a DSMS, both because they are not known in advance for the purposes of query optimization, identification of common sub-expressions across queries, etc., and more importantly because the correct answer to an ad-hoc query may require referencing data elements that have already arrived on the data streams (and potentially have already been discarded).

6.6 Issues in Data Stream Query Processing

Query processing in the data stream model of computation comes with its own unique challenges. In this section, we will outline what we consider to be the most interesting of these challenges, and describe several alternative approaches for resolving them.

6.6.1 Unbounded Memory Requirements

Since data streams are potentially unbounded in size, the amount of storage required to compute an exact answer to a data stream query may also grow without bound. Algorithms that use external memory are not well-suited to data stream applications since they do not support continuous queries and are typically too slow for real-time response. New data is constantly arriving even as the old data is being processed; the amount of computation time per data element must be low, or else the latency of the computation will be too high and the algorithm will not be able to keep pace with the data stream.

For this reason, we are interested in algorithms that are able to confine themselves to main memory without accessing disk.

6.6.2 Approximate Query Answering

When we are limited to a bounded amount of memory, it is not always possible to produce exact answers for the data stream queries; however, high-quality approximate answers are often acceptable in lieu of exact answers. Approximation algorithms for problems defined over data streams have been a fruitful research area in the algorithms community in recent years. This work has led to some general techniques for data reduction and synopsis construction, including: sketches, random sampling, histograms and wavelets. For example, recent work develops histogram-based techniques to provide approximate answers for *correlated aggregate queries* over data streams and Gilbert *et al.* present a general approach for building small space summaries over data streams to provide approximate answers for many classes of aggregate queries.

6.6.3 Sliding Windows

One technique for approximate query answering is to evaluate the query not over the entire past history of the data streams, but rather only over sliding windows of the recent data from the streams. For example, only data from the last week could be considered in producing query answers, with data older than 1 week being discarded. Imposing sliding windows on data streams is a natural method for approximation that has several attractive properties. Most importantly, it emphasizes recent data, which in the majority of real-world applications is more important and relevant than the old data: If one is trying in real-time to make sense of network traffic patterns, or phone call or transaction records, or scientific sensor data, then in general insights based on the recent past will be more informative and useful than insights based on stale data. In fact, for many such applications, sliding windows can be a requirement needed as a part of the desired query semantics explicitly expressed as a part of the user's query. A sliding window can be the most recent n elements of a stream, for some n , or it can be all the elements that arrived within the last t time units, for example, 1 month. If we regard each stream element as a tuple, we can treat the window as a relation and query it with any SQL query. Of course, the stream-management system must keep the window fresh, deleting the oldest elements as new ones come in.

Example 1

Suppose a user wanted to compute the average call length, but considering only the 10 most recent long-distance calls placed by each customer. The query can be formulated as follows:

```
SELECT AVG(S.minutes)
FROM Calls S [PARTITION BY S.customer id
ROWS 10 PRECEDING
WHERE S.type = "Long Distance"]
```


Example 2

A slightly more complicated example will be to return the average length of the last 1000 telephone calls placed by “Gold” customers:

```
SELECT AVG(V.minutes)
FROM (SELECT S.minutes
FROM Calls S, Customers T
WHERE S.customer id = T.customer id
AND T.tier = “Gold”)
V [ROWS 1000 PRECEDING]
```

Notice that in these examples, the stream of calls must be joined to the customers relation before applying the sliding window.

6.6.4 Batch Processing, Sampling and Synopses

Another class of techniques for producing approximate answers is to avoid looking at every data element and restrict query evaluation to some sort of sampling or batch processing technique. In batch processing, rather than producing a continually up-to-date answer, the data elements are buffered as they arrive, and the answer to the query is computed periodically. This is an approximate answer since it represents the exact answer at a point in the recent past rather than the exact answer at the present moment. This approach of approximation through batch processing is attractive because it does not cause any uncertainty about the accuracy of the answer, sacrificing timeliness instead. It is also a good approach when data streams are bursty.

Sampling is based on the principle that it is a futile attempt to make use of all the data when computing an answer, because data arrives faster than it can be processed. Instead, some data points must be skipped altogether, so that the query is evaluated over a sample of the data stream rather than over the entire data stream.

For some classes of data stream queries where no exact data structure with the desired properties exist, one can often design an approximate data structure that maintains a small *synopsis* or *sketch* of the data rather than an exact representation, and therefore, is able to keep computation per data element to a minimum.

6.6.5 Blocking Operators

A *blocking query operator* is a query operator that is unable to produce an answer until it has seen its entire input. Sorting is an example of a blocking operator, as are aggregation operators such as SUM, COUNT, MIN, MAX and AVG. If one thinks about evaluating the continuous stream queries using a traditional tree of query operators, where data streams enter at the leaves and final query answers

warned, if that too returned a positive result). Several variants of this general Bloom Filter exist with different capabilities.

6.8.2 The Bloom Filter

A **Bloom filter** is a space-efficient probabilistic data structure, conceived by Burton Howard Bloom in 1970, that is used to test whether an element is a member of a set. False positive matches are possible, but false negatives are not, thus a Bloom filter has a 100% recall rate. In other words, a query returns either “possibly in set” or “definitely not in set”. Elements can be added to the set, but not removed (though this can be addressed with a “counting” filter). The more elements that are added to the set, the larger the probability of false positives.

The Bloom filter for a set is much smaller than the set itself, which makes it appropriate for storing large amounts of data required when filtering on a data stream in main memory itself.

The advantage of this data structure is that it uses considerably less space than any exact method, but pays for this by introducing a small probability of error. Depending on the space available, this error can be made arbitrarily small.

Let us assume we want to represent n -element sets $S = s_1, s_2, \dots, s_n$ from a very large universe U , with $|U| = u \gg n$.

We want to support insertions and membership queries (as to say “Given $x \in U$ is $x \in S$?”) so that:

1. If the answer is No, then $x \in S$.
2. If the answer is Yes, then x may or may not be in S , but the probability that $x \notin S$ (false positive) is low.

Both insertions and membership queries should be performed in a constant time. A Bloom filter is a bit vector B of m bits, with k independent hash functions $h_1, \dots, h_k$ that map each key in U to the set $R_m = 0, 1, \dots, (m-1)$. We assume that each hash function h_i maps a uniformly at random chosen key $x \in U$ to each element of R_m with equal probability. Since we assume the hash functions are independent, it follows that the vector $[h_1(x), \dots, h_k(x)]$ is equally likely to be any of the m^k k -tuples of elements from R_m .

Algorithm

- Initially all m bits of B are set to 0.
- Insert x into S . Compute $h_1(x), \dots, h_k(x)$ and set

$$B[h_1(x)] = B[h_2(x)] = \dots = B[h_k(x)] = 1$$

- Query if $x \in S$. Compute $h_1(x), \dots, h_k(x)$.

If $B[h_1(x)] = B[h_2(x)] = \dots = B[h_k(x)] = 1$, then answer Yes, else answer No.

6.9 Counting Distinct Elements in a Stream

The **count-distinct problem**, known in applied mathematics as the **cardinality estimation problem**, is the problem of finding the number of distinct elements in a data stream with repeated elements. This is a well-known problem with numerous applications. The elements might represent IP addresses of packets passing through a router, unique visitors to a website, distinct elements in a large database, motifs in a DNA sequence, etc. In general, if stream elements are chosen from some universal sets, we would like to know how many different elements have appeared in the stream, counting from either the beginning of the stream or some known time in the past.

6.9.1 Count Distinct Problem

Let us first give a formal description of the problem in the following way:

Problem: Given a stream $X = \langle x_1, x_2, \dots, x_m \rangle \in [n]^m$ of values. Let F_0 be the number of distinct elements in X . Find F_0 under the following constraints on data stream algorithms.

Constraints: The constraints are as follows:

1. Elements in stream are presented sequentially and single pass is allowed.
2. Limited space to operate. Avoid swaps form secondary memory. Expected space complexity $O(\log(\min(n, m)))$ or smaller.
3. Estimation guarantees: With error $\varepsilon < 1$ and high probability.

For example consider the stream with instance, $\{a, b, a, c, d, b, d\}$ $F_0 = 4 = |\{a, b, c, d\}|$.

A naïve solution for this problem would be to use a counter $c(i)$ for each value i of the universe U , and would therefore require $O(1)$ processing time per item, but linear space. If the universe is $[1 \dots n]$, a bit vector of size n can be used as the synopsis, initialized to 0, where bit i is set to 1 upon observing an item with value i . However, in many cases, the universe is quite large (e.g., the universe of IP addresses is $n = 2^{32}$), making the synopsis much larger than the stream size M . Alternatively, the count F_0 can trivially be computed exactly in one pass through the data stream, by just keeping track of all the unique values observed in the stream. We can maintain a set of all the unique values seen, which takes $n \log_2 n$ bits because each value is $\log_2 n$ bits. However, F_0 can be as large as n , so again the synopsis is quite large. Over two decades ago, Flajolet and Martin presented the first small space distinct-values estimation algorithm called the Flajolet–Martin (FM) algorithm.

6.9.2 The Flajolet–Martin Algorithm

The principle behind the FM algorithm is that, it is possible to estimate the number of distinct elements by hashing the elements of the universal set to a bit-string that is sufficiently long. This means that the length of the bit-string must be such that there are more possible results of the hash function than there are elements of the universal set.

Before we can estimate the number of distinct elements, we first choose an upper boundary of distinct elements m . This boundary gives us the maximum number of distinct elements that we might be able to detect. Choosing m to be too small will influence the precision of our measurement. Choosing m that is far bigger than the number of distinct elements will only use too much memory. Here, the memory that is required is $O[\log(m)]$.

For most applications, 64-bit is a sufficiently large bit-array. The array needs to be initialized to zero. We will then use one or more adequate hashing functions. These hash functions will map the input to a number that is representable by our bit-array. This number will then be analyzed for each record. If the resulting number contained k trailing zeros, we will set the k th bit in the bit array to one. Finally, we can estimate the currently available number of distinct elements by taking the index of the first zero bit in the bit-array. This index is usually denoted by R . We can then estimate the number of unique elements N to be estimated by $N = 2^R$. The algorithm is as follows:

Algorithm

- Pick a hash function h that maps each of the n elements to at least $\log_2 n$ bits.
- For each stream element a , let $r(a)$ be the number of trailing 0's in $h(a)$.
- Record $R =$ the maximum $r(a)$ seen.
- Estimate $= 2^R$.

Example 4

$r(a)$ = position of first 1 counting from the right; say $h(a) = 12$, then 12 is 1100 in binary, so $r(a) = 2$

Example 5

Suppose the stream is 1, 3, 2, 1, 2, 3, 4, 3, 1, 2, 3, 1 ... Let $h(x) = 3x + 1 \pmod{5}$.

- So the transformed stream (h applied to each item) is 4, 5, 2, 4, 2, 5, 3, 5, 4, 2, 5, 4
- Each of the above element is converted into its binary equivalent as 100, 101, 10, 100, 10, 101, 11, 101, 100, 10, 101, 100
- We compute $r(a)$ of each item in the above stream: 2, 0, 1, 2, 1, 0, 0, 0, 2, 1, 0, 2
- So $R =$ maximum $r(a)$, which is 2. Output $2^2 = 4$.

disk, or there are so many streams that windows for all cannot be stored. For example, we are processing 1 billion streams and $N = 1$ billion; thus, we cannot afford to store N bits. This may be true if for instance we are trying to get counts of a particular concept on Twitter to see if its count indicates that is a trending topic currently. As another example, consider a network management application, where a large number of data packets pass through a router every second and we need to compute various statistics for this stream. However, in most applications, it suffices to produce an approximate answer. It is interesting to observe why naive schemes do not suffice for producing approximate answers with low memory requirement. For instance, consider the scheme in which we maintain a simple counter that is incremented upon the arrival of a data element, which is 1. The problem is that an old data element expires at every time instant, but we have no way of knowing whether that was a 0 or 1 and whether we should decrement the counter. Another simple method is random sampling. Just maintaining a sample of the window elements will fail in the case where the 1s are relatively sparse. The sampling method will also fail when the 1s may not arrive in a uniform manner.

6.10.2 The Datar–Gionis–Indyk–Motwani Algorithm

Datar–Gionis–Indyk–Motwani presented an elegant and simple algorithm called DGIM, which uses $O(\log_2 N)$ bits to represent a window of N bits, and enables the estimation of the number of 1s in the window with an error of no more than 50%.

There are certain constraints that must be satisfied for representing a stream by buckets using the DGIM algorithm.

1. The right end of a bucket always starts with a position with a 1.
2. Number of 1s must be a power of 2. That explains the $O(\log \log N)$ bits needed for storing the number of 1s.
3. Either **one** or **two** buckets with the same **power-of-2 number** of 1s exists.
4. **Buckets do not overlap in timestamps.**
5. **Buckets are sorted by size.** Earlier buckets are not smaller than later buckets.
6. Buckets disappear when their end-time is $> N$ time units in the past.

Figure 6.4 indicates an example of a bit stream bucketized using DGIM constraints.

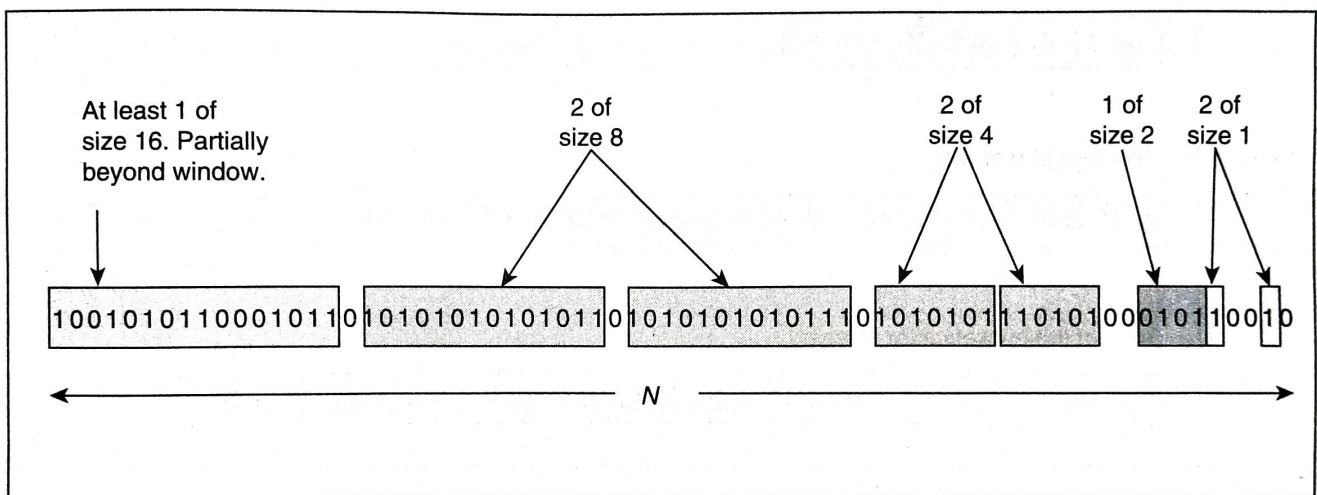


Figure 6.4 Example of a bit stream bucketized using DGIM constraints.

As discussed earlier, each bucket can be represented by $O(\log N)$ bits. For a window of length N , there can be maximum N 1s. Suppose the largest bucket is of size 2^j . Thus j cannot exceed $\log_2 N$. Thus, there are at most two buckets of all sizes from $\log_2 N$ down to 1, and no buckets of larger sizes. So we can conclude that there are $O(\log N)$ buckets. Since each bucket can be represented in $O(\log N)$ bits, the total space required for all the buckets representing a window of size N is $O(\log^2 N)$.

6.10.3 Query Answering in DGIM Algorithm

At any time instant, we may produce an estimate of the number of active 1s as follows. For all but the last bucket, we add the number of 1s that are in them. For the last bucket, let C be the count of the number of 1s in that bucket. The actual number of active 1s in this bucket could be anywhere between 1 and C , and so we estimate it to be $C/2$.

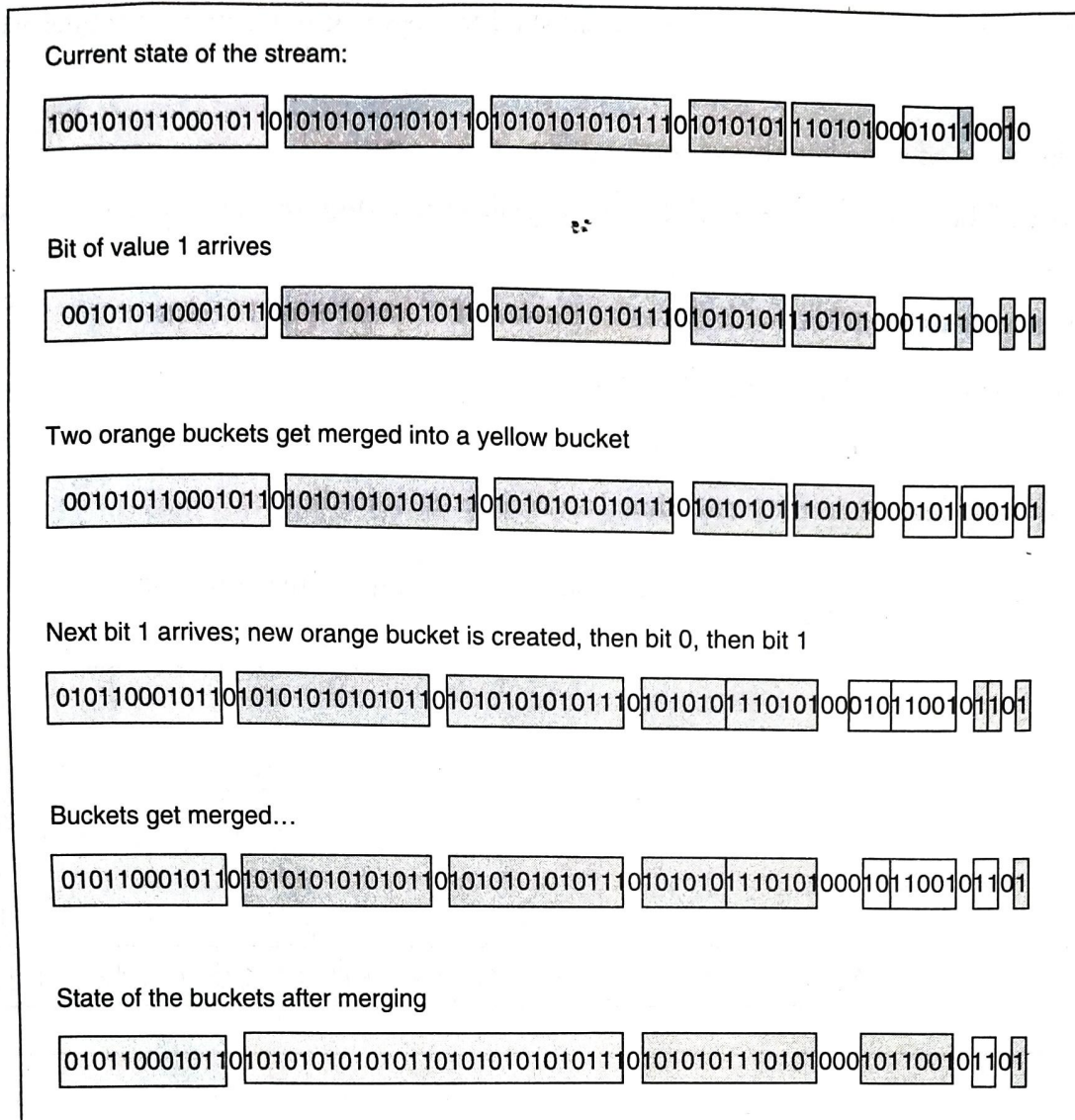


Figure 6.5 Merging Buckets.

For example, for the stream instance shown in Fig. 6.5, the number 1s can be estimated as follows:

1. Estimated: $1 + 1 + 2 + 4 + 4 + 8 + 8 + (1/2) \cdot 16 = 36$
2. Exact: $1 + 1 + 2 + 4 + 4 + 8 + 8 + 7 = 35$

In general to estimate the number of 1s in the last k ($1 \leq k \leq N$) bits:

1. Determine bucket b with the earliest timestamp that includes at least some of the N most recent bits.
2. Sum the sizes of all buckets to the right of b . (Recall that "size" means the number of 1s in the bucket).
3. Final estimate: Add half the size of bucket b .

Since we do not know how many 1s of the last bucket are still within the wanted window we are approximating it as half the bucket size.

Example 7

Suppose the stream is that of Fig. 6.5 and $k = 10$. Then the query asks for the number of 1s in the ten rightmost bits, which happen to be 0110010110. Let the current timestamp (time of the rightmost bit) be t . Then the two buckets with one 1, having timestamps $t - 1$ and $t - 2$ are completely included in the answer. The bucket of size 2, with timestamp $t - 4$, is also completely included. However, the rightmost bucket of size 4, with timestamp $t - 8$ is only partly included. We know it is the last bucket to contribute to the answer, because the next bucket to its left has timestamp less than $t - 9$, and thus, is completely out of the window. On the other hand, we know the buckets to its right are completely inside the range of the query because of the existence of a bucket to their left with timestamp $t - 9$ or greater.

Our estimate of the number of 1s in the last ten positions is thus 6. This number is the two buckets of size 1, the bucket of size 2, and half the bucket of size 4 that is partially within range. Of course the correct answer is 5.

Suppose the last bucket has size 2^r , then by assuming 2^{r-1} (i.e., half) of its 1s are still within the window, we make an error of at most 2^{r-1} . Since there is at least one bucket of each of the sizes less than 2^r , the true sum is at least $1 + 2 + 4 + \dots + 2^{r-1} = 2^r - 1$. Thus, error can at most be 50%.

1. **Case 1:** Estimate is less than exact answer. Worst case: all 1s of B are in range of the query but only half are included. Thus, estimate is at least 50% of the exact answer.
2. **Case 2:** Estimate is greater than exact answer. Worst case: only the rightmost bit of B (2^r) is in range and only one bucket of each size smaller than B exists.
3. **Exact:** $1 + 2^{r-1} + 2^{r-2} + \dots + 1 = 2^r$
4. **Estimated:** $2^{r-1} + 2^{r-1} + 2^{r-2} + \dots + 1 = 2^r + 2^{r-1} - 1$

Thus, the estimate is at most 50% greater than the exact answer always.

6.10.4 Updating Windows in DGIM Algorithm

In a data stream newer bits are arriving at every instant of time. Suppose we have a window of length N properly represented by buckets that satisfy the DGIM constraints. When a new bit comes in, we may need to modify the buckets, so they continue to represent the window and continue to satisfy the DGIM conditions. The following steps describe the Update Algorithm for sliding Windows satisfying DGIM constraints.

1. When a new bit comes in, drop the last (oldest) bucket if its end-time is prior to N time units before the current time
2. **2 cases:** Current bit is 0 or 1
3. **If the current bit is 0: no other changes are needed**
4. **If the current bit is 1:**
 - Create a new bucket of size 1, for just this bit. Make the **End timestamp** indicate the **current time**.
 - If there was only one bucket of size 1 before, then nothing more needs to be done. But if there is now **three buckets of size 1**, then **the oldest two must be combined into a new into a bucket of size 2**.
 - If this results in **three buckets of size 2**, combine the oldest two into a bucket of size 4
 - **And so on ...**

To combine any two adjacent buckets of the same size, replace them by one bucket of twice the size. The timestamp of the new bucket is the timestamp of the rightmost (later in time) of the two buckets.

6.11 Decaying Windows

Pure sliding windows are not the only way by which the evolution of data streams can be taken into account during the mining process. A second way is to introduce a decay factor into the computation. Specifically, the weight of each transaction is multiplied by a factor of $f < 1$, when a new transaction arrives. The overall effect of such an approach is to create an exponential decay function on the arrivals in the data stream. Such a model is quite effective for evolving data stream, since recent transactions are counted more significantly during the mining process. Specifically, the decay factor is applied only to those itemsets whose counts are affected by the current transaction. However, the decay factor will have to be applied in a modified way by taking into account the last time that the itemset was touched by an update. This approach works because the counts of each itemset reduce by the same decay factor in each iteration, as long as a transaction count is not added to it. Such approach is also applicable to other mining problems where statistics are represented as the sum of decaying values.

We discuss a few applications of decaying windows to find interesting aggregates over data streams.

6.11.1 The Problem of Most-Common Elements

Suppose we have a stream whose elements are the movie tickets purchased all over the world, with the name of the movie as part of the element. We want to keep a summary of the stream that is the most

- (d) In an e-voting system checking whether an attempt is being made to cast a duplicate vote.
5. A Bloom filter with $m = 1000$ cells is used to store information about $n = 100$ items, using $k = 4$ hash functions. Calculate the false positive probability of this instance. Will the performance improve by increasing the number of hash functions from 4 to 5. Explain your answer.
6. Employ the DGIM algorithm. Shown below is a data stream with $N = 24$ and the current bucket configuration. New elements enter the window at the right. Thus, the oldest bit of the window is the left-most bit shown.
- ```

1 0 1 0 1 1 0 0 0 1 0 1 1 1 0 1 1 0 0
 1 0 1 1 0

```
- (a) What is the largest possible bucket size for  $N = 24$ ?
- (b) Show one way of how the above initial stream will be divided into buckets
- (c) What is the estimate of the number of 1's in the latest  $k = 14$  bits of this window?
- (d) The following bits enter the window, one at a time: 1, 0, 1, 0, 1, 0, 1, 1. What is the bucket configuration in the window after this sequence of bits has been processed by DGIM?
- (e) After having processed the bits from (c), what is now the estimate of the number of 1's in the latest  $k = 14$  bits of the window?
7. We wish to use the Flajolet–Martin Algorithm to count the number of distinct elements in a stream. Suppose that there 10 possible elements, 1, 2, ..., 10 that could appear in the stream, but only four of them have actually appeared. To make our estimate of the count of distinct elements, we hash each element to a 4-bit binary number. The element  $x$  is hashed to  $3x + 7$  (modulo 11). For example, element 8 hashes to  $3 * 8 + 7 = 31$ , which is 9 modulo 11 (i.e., the remainder of  $31/11$  is 9). Thus, the 4-bit string for element 8 is 1001.
- A set of four of the elements 1 through 10 could give an estimate that is exact (i.e., 4 distinct elements have appeared), or too high, or too low. List a set of four elements that gives the exact estimate. Briefly describe why this set gives the exact estimate of having seen 4 distinct elements.
8. Suppose a stream consists of the integers 2, 1, 6, 1, 5, 9, 2, 3, 5. Let the hash functions all be of the form  $h(x) = ax + b \bmod 16$  for some  $a$  and  $b$ . You should treat the result as a 4-bit binary integer. Determine the tail length for each stream element and the resulting estimate of the number of distinct elements if the hash function is:
- (a)  $h(x) = 2x + 3 \bmod 16$ .
- (b)  $h(x) = 4x + 1 \bmod 16$ .
- (c)  $h(x) = 5x \bmod 16$ .

## Programming Assignments

1. Implement the Flajolet–Martin estimate for counting distinct elements from a large data stream using Map reduce. Use the Bloom filter as a data structure.
2. Use the above algorithm to identify the number of distinct customers from any e-commerce dataset (Amazon dataset from